

Swift Codebase Documentation Report

Project Overview

- **Project Name:** Swift
- **Purpose:** A comprehensive, real-time logistics and food delivery platform specifically designed for campus ecosystems.
- **Problem it Solves:** Bridges the gap between students/staff and local campus vendors, providing a structured, real-time ordering experience optimized for the unique constraints of a university (e.g., delivery to classrooms, quiet zones, and scheduled breaks).
- **Target Users:**
 - **Students/Staff:** Order food, track deliveries, and manage class-linked schedules.
 - **Vendors:** Manage order queues, prep times, and staff permissions.
 - **Admirators:** Oversee platform health, moderate users/vendors, and manage finances.
- **Tech Stack:**
 - **Frontend:** Flutter (3.10+) for iOS, Android, and Web.
 - **Backend:** Node.js with Fastify (TypeScript).
 - **Database:** Supabase (PostgreSQL) with Row Level Security (RLS) and Realtime.
 - **Payments:** Razorpay integration.
 - **Infrastructure:** Vercel for backend hosting, Supabase for DB/Auth/Storage.

Architecture

- **High-Level Architecture:** Monorepo containing four main applications (Backend, User App, Vendor App, Admin App) communicating via a central REST API.
- **Folder Structure:**
 - `backend/`: Fastify server, modularized by features (auth, orders, admin, etc.).
 - `user_app/`: Consumer-facing Flutter application.
 - `vendor_app/`: Operations-facing Flutter application for merchants.
 - `admin_app/`: Governance-facing Flutter application for platform operators.
 - `supabase/`: Database schema, migrations, and RLS policy definitions.
- **Design Patterns:**
 - **Backend:** Controller-Route-Service pattern. Uses custom Fastify plugins for authentication and error handling.
 - **Frontend:** Riverpod for state management, `go_router` for declarative navigation, and Repository pattern for API abstraction.
- **Data Flow:**
 1. **Frontend** initiates a request (e.g., `Place Order`).
 2. **Backend Middleware** validates the JWT via Supabase Auth and checks RBAC/Block status.
 3. **Controller** processes the request, communicating with **Supabase Service** to interact with Postgres.
 4. **Postgres RLS** ensures the user can only access/modify their own data.
 5. **Realtime Subscriptions** notify the Vendor App of new orders and the User App of status updates.

Features

Core Features

- **Multi-App Ecosystem:** Dedicated interfaces for Users, Vendors, and Admins.
- **Real-time Order Tracking:** Live status updates from `pending` to `completed` with rolling ETA estimates.
- **Campus-Aware Delivery:** Deliveries mapped to specific `campus_buildings` and `delivery_zones`.
- **Class Session Integration:** Users can link orders to their class schedule (`class_sessions`) for delivery to classrooms.
- **Integrated Payments:** Secure checkout via Razorpay.

Secondary Features

- **In-App Chat:** Support rooms for handling order issues and operation follow-ups.
- **Promo Management:** Robust promotion engine with validation against order totals and usage limits.
- **Vendor Pacing:** Metadata-driven prep-time assistance to help vendors manage kitchen load.
- **Review System:** Five-star rating and comment system per order/vendor.

Internal Features

- **Contract Registry:** Backend exposes `/api/v1/contracts/registry` as a source of truth for all API shapes.
- **Audit Logging:** Mandatory "reason" capture for sensitive admin actions (e.g., blocking a user).
- **Device Trust:** Admin app enforces posture checks (`Trusted` vs `Untrusted`) for security.

API Documentation

Detailed reference available in `[API_REFERENCE.md](file:///c:/project/food/API_REFERENCE.md)`.

Endpoint: `/api/v1/orders`

- **Method:** POST
- **Description:** Places a new food order.
- **Authentication:** Yes (Bearer Token)
- **Request Body:**

```
{
  "vendor_id": "uuid",
  "total_amount": 150.0,
  "items": [
    { "id": "uuid", "quantity": 2, "price": 75.00 }
  ],
  "delivery_mode": "class",
  "delivery_building_id": "uuid",
  "delivery_room": "B-201",
  "quiet_mode": true
}
```

- **Response:** 201 Created with order details and initial ETA.

Endpoint: `/api/v1/contracts/registry`

- **Method:** GET
- **Description:** Returns versioned contract metadata for all endpoints.
- **Authentication:** No

Authentication & Security

- **Mechanism:** JWT-based session management provided by Supabase Auth.
- **Middleware:** Custom `authMiddlewarePlugin` in Fastify extracts the `sub` (`user_id`) and `role` from the JWT.
- **RBAC:**
 - `user`: Can create orders, manage their own cart, and view their notifications.
 - `vendor`: Can manage menus, update order statuses, and view vendor stats.
 - `admin`: Full platform access, moderation, and finance visibility.
- **Governance:** Sensitive actions (blocking, rejecting) require a minimum 10-character `reason` stored in `admin_logs`.
- **Database Security:** Row Level Security (RLS) is enabled on all tables, ensuring cross-tenant isolation at the database level.

Database Design

- **Type:** PostgreSQL (Supabase)
- **Key Tables:**
 - `users`: Profile data and roles.
 - `vendors`: Merchant details and approval status.
 - `orders`: Central transaction record with delivery and ETA metadata.
 - `menu_items`: Product details, pricing, and availability.
 - `campus_buildings`: Geographical delivery points.
 - `admin_logs`: Audit trail for moderation actions.
- **Relationships:** Multi-level hierarchy (Vendor -> Menu -> MenuItem). Orders link Users to Vendors with a junction table for Items.

Data Flow (Sequence Flow)

1. **Order Lifecycle:**
User (Checkout) -> Backend (Create Order + Payment) -> Postgres (RLS Write) -> Vendor App (Realtime Notification) -> Vendor (Accepted -> Preparing -> Ready) -> User App (Realtime Update).
2. **Support Lifecycle:**
User (Create Ticket) -> Backend (Create Room) -> Admin (Join Room) -> Realtime Chat (Supabase Realtime).

Code Structure & Important Files

- **Backend Entry:** `[src/index.ts](file:///c:/project/food/backend/src/index.ts)`
- **App Wiring:** `[src/app.ts](file:///c:/project/food/backend/src/app.ts)` (Middleware, Error Handling).
- **Supabase Migration:** `[supabase/schema.sql](file:///c:/project/food/supabase/schema.sql)`
- **User App Entry:** `[user_app/lib/main.dart](file:///c:/project/food/user_app/lib/main.dart)`
- **Vendor App Entry:** `[vendor_app/lib/main.dart](file:///c:/project/food/vendor_app/lib/main.dart)`
- **Admin App Entry:** `[admin_app/lib/main.dart](file:///c:/project/food/admin_app/lib/main.dart)`

Deployment

- **Local Development:**
 1. Clone repo.
 2. `npm install` in backend.
 3. Run `supabase start` or use a cloud project.
 4. Run `npm run dev` in backend.
 5. Run `Flutter run` in the respective app directories.
- **Production:**
 - Backend: Vercel or Node.js Container.
 - Apps: Built as APK/IPA and hosted on Play Store/App Store.
 - Database: Supabase production project.

Performance & Scalability

- **Optimizations:**
 - **Realtime Sync:** Leverages Supabase Realtime instead of polling for order updates.
 - **Caching:** `user_app` uses local cache for cart and menus with backend sync fallback.
 - **Pagination:** All admin list endpoints implement standardized pagination (`page`, `limit`).
 - **Bottlenecks:**
 - Complex PostgreSQL joins in `stats/dashboard` endpoints (addressing via materialized views/caching).
 - Supabase Realtime connection limits on free tiers.
-

Known Issues / Technical Debt

- **Validations:** Some legacy endpoints still accept older field aliases (e.g., `item_id` vs `menu_item_id`) for backward compatibility.
 - **Mobile Analysis:** Flutter apps have some linter warnings related to `deprecated_member_use` in older SDK versions.
 - **API Redundancy:** Both `/health` and `/api/health` exist to support transition between routing layers.
-

Improvement Suggestions

- **Architecture:** Implement a shared Dart package for models used across all 3 Flutter apps to ensure type safety.
 - **Dev Ops:** Automate APK/IPA builds and deployment via GitHub Actions (Fastlane integration).
 - **Feature:** Add "Group Ordering" for students in the same dorm/building to reduce delivery overhead.
-

Summary

- **Final Evaluation:** The Swift codebase is highly structured, secure, and production-ready. Its use of RLS and modular architecture makes it resilient and scalable.
- **Readiness Level:** **Production** (Scalable).